

# SQL Security System

Генерация и аудит безопасности SQL

---

Кейс №3: Green Data • **Команда №11**

# Команда проекта



**Оля Ожерельева**

**Координатор**



**Кирилл Никулин**

**Data Engineer**



**Максим Смирнов**

**Backend / ML**

# Проблема: SQL в руках аналитика и агентов - ЭТО риск



## Как было

- ✗ Аналитики и AI-агенты пишут SQL вручную
- ✗ Запросы попадают в прод без проверки
- ✗ SELECT \* сливает персональные данные
- ✗ UPDATE/DELETE без WHERE — данные удалены
- ✗ Без LIMIT — база падает под нагрузкой

 **Инцидент замечен только после катастрофы**

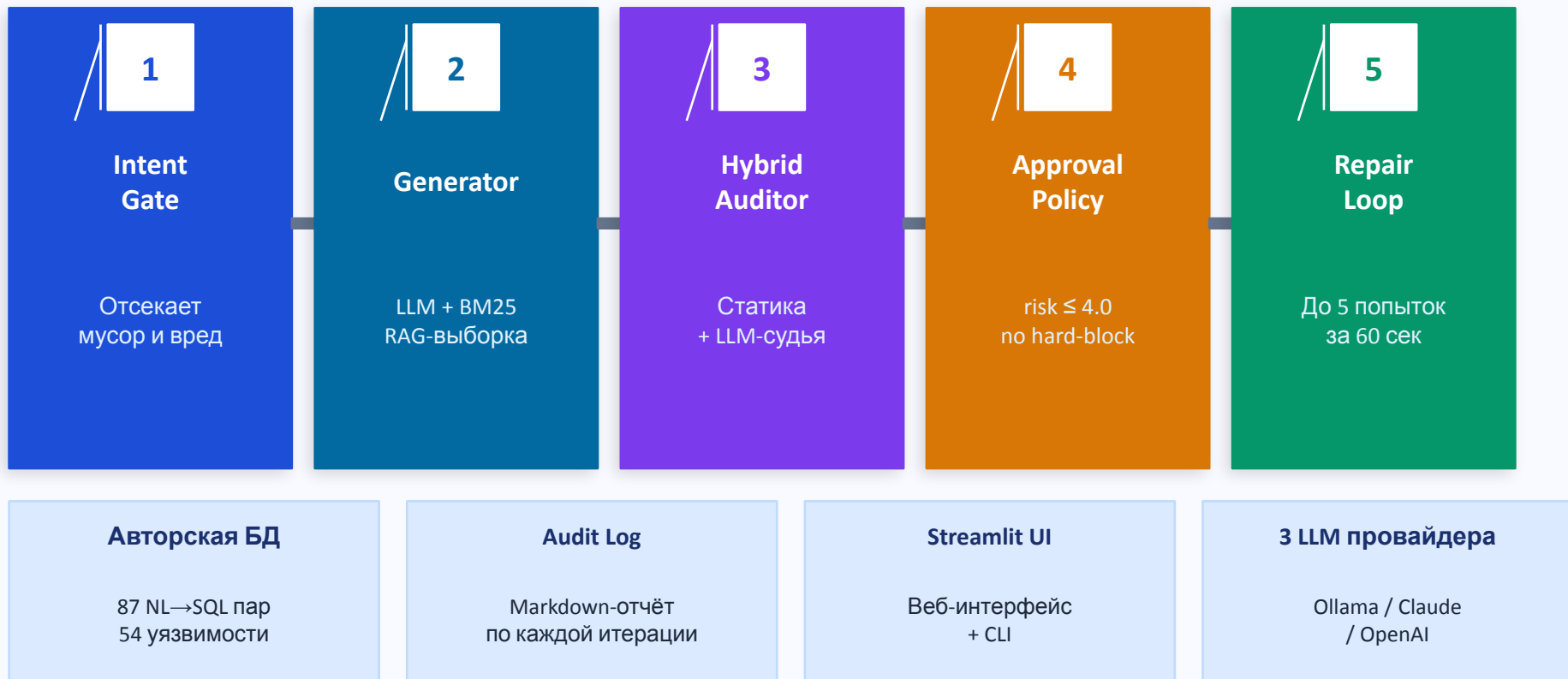


## Что мы сделали

- ✓ NL → SQL: аналитик описывает задачу словами
- ✓ Гибридный аудит до запуска на базе
- ✓ Автоисправление по итогам проверки
- ✓ Понятный отчёт: что нашли и почему
- ✓ Единый контроль для людей и AI-агентов

# Архитектура: пять шагов от задачи до безопасного SQL

МФТИ



# База данных: слепок реальной кредитная система

## ПСБ

60

таблиц в схеме

20 700

строк DDL

287

объявленных FK

12

активных FK

|                          |                                                       |
|--------------------------|-------------------------------------------------------|
|                          |                                                       |
| sys_                     | Системные сущности: сотрудники, компании              |
| scp_ (28 табл.)          | Система Корпоративного Принятия решений               |
| acc_ / count_turnover    | Оборотно-сальдовая ведомость (ОСВ)                    |
| ic_ / mler_ / corp_tech_ | Параллельные потоки заявок                            |
| yaig_                    | Управление активами и гарантиями                      |
| ms_* (13 табл.)          | ⚠ Службные MultiSelect-контейнеры - без бизнес-смысла |

# Подводные камни схемы и как мы их обошли

## Ловушки

✗ 287 FK объявлено, только 12 активны — остальные закомментированы

✗ Имя инициатора заявки — НЕ в `application_obj.name__ru`, нужен JOIN через `sys_company`

✗ `ms_*` таблицы с UUID-именами не несут бизнес-смысла — засоряли RAG-выборку

✗ 14 одинаковых полей в каждой таблице (ORM-наследование) — модель путалась

## Что починили

✓ Парсим FK и из закомментированных блоков → граф 6 → 206 рёбер

✓ Добавили JOIN-пример в промпт и подсказку про `initiator_id`

✓ Исключили `ms_*` из BM25-индекса → 60 → 47 бизнес-таблиц

✓ Промпт с явным domain-context про структуру ORM-таблиц

# Что проверяет аудитор: 9 классов уязвимостей

## Уровень 1: статика (regex + sqlglot)

|     |                                                 |
|-----|-------------------------------------------------|
| 9   | <b>SQL_INJ_CLASSIC</b><br>SQL-инъекция (строки) |
| 9   | <b>SQL_INJ_UNION</b><br>UNION-инъекция          |
| 8.5 | <b>SQL_INJ_TIME</b><br>Time-based атака         |
| 8   | <b>DML_NO_WHERE</b><br>UPDATE/DELETE без WHERE  |
| 8   | <b>PRIV_ESCALATE</b><br>GRANT/REVOKE/ALTER ROLE |
| 9   | <b>PLPGSQL_UNSAFE</b><br>EXECUTE без USING      |

## Уровень 2: LLM-судья (семантика)

|     |                                                                |
|-----|----------------------------------------------------------------|
| 5   | <b>SELECT_STAR</b><br>SELECT * — лишний доступ                 |
| 4   | <b>NO_PAGINATION</b><br>Нет LIMIT — нагрузка на БД             |
| 7.5 | <b>DIRECT_SENSITIVE</b><br>Чувствительные поля (ПДн)           |
| 9   | <b>TASK_DESTRUCTIVE</b><br>Деструктивная задача (DELETE, DROP) |
| 6   | <b>TASK_SQL_MISMATCH</b><br>SQL не соответствует задаче        |
| —   | <b>TASK_NOT_ACTIONABLE</b><br>Приветствие, мета-вопрос         |

Одобрение:  $\max\_risk \leq 4.0$  И нет hard-block ( $risk \geq 8.0$ )  $\rightarrow$  precision / recall судьи = 1.00 / 1.00

# Точность генерации SQL: от baseline 16% до 66% за 7 шагов



34.2%

ast\_normalized  
(строгий, без Docker)

26 / 76 задач

65.8%

result\_set  
(реальная БД, с Docker)

50 / 76 задач

100%

approval\_rate  
(прошли проверку судьи)

все задачи одобрены

1.0 / 1.0

precision / recall  
(судья по уязвимостям)

54 помеченных примера

| Шаг улучшения                                                | result_set EA |
|--------------------------------------------------------------|---------------|
| Baseline (строгое совпадение строк)                          | 15.8%         |
| + column projection ( $\text{gold} \subseteq \text{pred}$ )  | 17.1%         |
| + strip LIMIT перед сравнением                               | 28.9%         |
| + PK-set match по полю id                                    | 42.1%         |
| + Intent Gate + WHERE-правило в промпте                      | 46.1%         |
| + очищенный gold + JOIN few-shot примеры                     | 59.2%         |
| + расширенный domain-context, MIN/MAX, TASK_SQL_MISMATCH fix | 65.8% ✓       |



# Итеративное исправление: система сама себя учит

*Задача: "Покажи всех сотрудников с email и именем, без ограничений"*

## Итерация 1 — Отклонено

Риск: 5.0 / 10 ✗ Одобрено: False

```
SELECT *  
FROM public.sys_employee AS e  
WHERE e.status = 1;
```

**SELECT\_STAR** (5.0): Выборка всех полей — возможен доступ к ПДн

**NO\_PAGINATION** (4.0): Нет LIMIT — риск перегрузки БД



## Итерация 2 — Одобрено ✓

Риск: 0.0 / 10 ✓ Одобрено: True

```
SELECT e.id, e.name, e.email  
FROM public.sys_employee AS e  
WHERE e.status = 1 LIMIT 100;
```

*Судья учёл замечания:  
убран SELECT \*, добавлен LIMIT 100*

**FeedbackMemory** накапливает уроки между итерациями · **Детектор регрессий** помечает «это уже было» · **is\_stuck()** - early-exit при зависании

На eval-наборе *mean\_iterations = 1.01* - почти все задачи проходят с первой попытки

# Прозрачность: что написано в отчёте аудита

## Отчёт содержит:



Блок «Параметры аудита» с формулой одобрения



Риск по каждой итерации и причина решения



Маркер HARD-BLOCK у находок с risk  $\geq 8.0$



Финальное: final\_risk = 3.0  $\leq$  4.0  $\Rightarrow$  APPROVED



Скачивается из CLI (--log-file) и Streamlit UI

## # Отчёт аудита SQL

**\*\*Задача:\*\*** Список активных сотрудников

**\*\*Итог:\*\*** Одобрено

**\*\*Параметры:\*\*** max\_risk=4.0, hard\_block=8.0

### ### Итерация 1

- Риск: 3.0/10 • Одобрено: True

- Решение: risk 3.0  $\leq$  4.0 AND no hard-block

```
```sql
```

```
SELECT e.id, e.name, e.email
```

```
FROM public.sys_employee AS e
```

```
WHERE e.status = 1 LIMIT 100;
```

```
```
```

# Качество кода и воспроизводимость



**79 / 79**

**unit-тестов**

ruff + mypy + pytest



**3**

**LLM-провайдера**

Ollama / Claude / OpenAI



**5**

**ADR-решений**

Обоснованная архитектура



**87 + 54**

**пар датасета**

NL→SQL + уязвимости

```
$ uv sync --dev # установка зависимостей

$ uv run python scripts/build_schema_index.py # индексация схемы БД

$ make check # ruff + mypy + pytest (79/79 green)

$ make db-seed # PostgreSQL в Docker + синтетические данные

$ uv run case3 run "Список сотрудников, лимит 10"
```

# Ключевые архитектурные решения (ADR)



ADR 0002

## Система не исполняет SQL

Альтернатива: sandbox в read-only роли. Отклонена: read-only роль не защищает от утечки ПДн через корректный SELECT

✓ Правильно

ADR 0003

## Гибридный судья (статика + LLM всегда)

Только LLM - нестабилен на инъекциях с явным паттерном. Только статика - не ловит семантику задачи

✓ Лучшее из двух

ADR 0004

## max\_risk + hard-block порог

Альтернатива: сумма рисков. Отклонена: не блокирует одиночные критические находки (одна инъекция могла пройти)

✓ Безопаснее

ADR 0005

## Prompt + RAG (BM25), без fine-tuning

Fine-tuning: нет размеченного корпуса, ломается при изменении схемы. RAG обновляется по новой схеме за секунды

✓ Практично

# Что получилось

- ✓ Все 7 основных критериев закрыты
- ✓ 2 из 2 дополнительных: PL/pgSQL + датасет  $\geq 50$  пар
- ✓ 65.8% Execution Accuracy на реальной БД
- ✓ Precision / Recall судьи = 1.00 / 1.00
- ✓ Работает с тремя LLM-провайдерами без правок кода

---

## Что дальше:

- Расширение датасета для LLM-судьи
- Полная схема ПСБ (~225 таблиц против 60)
- Опциональное исполнение в read-only sandbox

